FORJA — 需求锻造平台

规格入炉，交付出炉 · Spec 驱动的 AI 研发流水线

版本：v2.2.0（含 Python 第三栈 / 模板参数化 / UAT 回炉闭环）
文档生成：2026-09-19 | 配套：GitHub Pages 在线文档

一、项目背景

软件团队的需求、规格、实现、测试、交付长期分散在多个工具中：需求在文档里、规格在沟通中、 实现在代码里、验收在线下。结果是**知识随人员流失、质量靠事后补救、交付靠手工拼装**， 边际成本不降反升。

FORJA（锻造坊）把这一链路收敛为一个平台：以 **Spec 为唯一事实源**、 以 **Agent 为执行引擎**、以 **门禁与签核为质量闸口**， 实现"需求入堆 → 结构化规格 → AI 锻造 → 构建镜像 → 交付包 → 客户部署 → 验收回炉"的全闭环。

3	≤3 轮	5 件套	1 键
技术栈模板（Go / Node / Python）	失败自动修复	标准交付包	UAT 失败回炉

解决的核心问题

1. **规格与实现脱节**：四层 Spec（BS/IS/DS/TS）+ 回归约束自动注入，AI 严格按规格产出。
2. **质量靠人盯**：静态检查 / 构建 / 单测 / 覆盖率 / 安全扫描五道门禁，失败自动重锻。
3. **交付不可复制**：交付包含 compose / 部署手册 / Release Notes / 镜像，客户自部署。
4. **验收不闭环**：UAT 不通过一键回炉（幂等），携带交付上下文生成修复任务。

二、技术架构

2.1 总体分层

层	组成
前端	React 19 + TypeScript + Vite；TanStack Query/Router；Tailwind + shadcn/ui；SSE 实时流水线
后端	Go 1.24 + Fiber；sqlc/pgx 数据访问；asynq + Redis 异步队列；三个入口（api / worker / storage）
数据	PostgreSQL 16（pgvector 语义检索）；Redis 7（会话 / 队列 / 缓存）
研发流水线	Gitea + Gitea Actions（DinD runner + socket-proxy 通道）
Agent	OpenCode Server（OpenAI 兼容 Provider，可切私有化模型）；按项目隔离的"炉"容器

2.2 核心链路（数据视角）

`Inbox 原料` → `Spec（四层 + refs/anchors）` → `Task（分支隔离）` → `Agent Execution（attempt/diff/tokens）` → `Run/Job（门禁回流）` → `Artifact（镜像 tar）` → `Delivery（交付包）` → `UAT` → `Reforge（幂等回炉）`

2.3 关键设计

- **仓库即真相**：项目代码在 Git，流水线与制品回填写本地台账，页面不依赖 Gitea 存活。
- **分支隔离锻造**：`forge/task-<id>-a<n>`，签核后才 `--no-ff` 合并并打标签。
- **门禁动态化**：按栈分派（Go vet/test、Node test/lcov、Python pytest/lcov），阈值棘轮可配。
- **模板参数化**：`template.yaml` 声明 enum/bool 选项（≤6 项、白名单），建项渲染落仓。
- **安全基线**：模型 key AES-GCM 加密 + 双钥轮换；容器非 root、只读 rootfs、限权 Docker 通道。



项目详情：成员 / 规格 / 任务 / 交付 / 成本 / 动态一体



锻造面板：阶段步进 + attempt 轮次 + 门禁摘要 + 签核

三、应用场景

3.1 企业内部研发提效

PM 提需求、架构师定规格、开发者关注"为什么失败"，重复劳动（脚手架、单测、门禁、打包） 交给平台与 Agent，人力集中在判断与设计。

3.2 外包 / 项目制交付

每个客户一个项目仓 + 一套交付包；验收反馈（UAT）直接回炉为修复任务， "交付 → 验收 → 修复 → 再交付"闭环可追溯，成本按项目/周期分包结算。

3.3 多技术栈团队

Go / Node / Python 三栈统一流程：同一套 Spec、门禁、交付与治理语义， 模板可参数化复用于同栈多形态（如 Python 版本 / 健康路径 / 文档开关）。

3.4 自举（Dogfooding）

平台自身即以 FORJA 管理：规格、锻造、门禁、交付、审计全链路作用于自身仓库， "用自己锻造自己"保证流程可用性与持续演进。



任务看板：4+2 列、依赖 DAG、异常竖签



构建部署：流水线明细 + 部署记录台账

四、创新点

创新点	说明
Spec 即契约的锻造闭环	四层 Spec + Clarify 门禁 + 回归约束自动注入，Agent 产出可校验、可追溯，签核即合并
失败自愈的锻造流水线	门禁失败携带上下文自动重锻（≤3 轮），升级人工时保留每轮产物与摘要
模板参数化 + 多栈统一	组织级模板仓（版本 pin）+ enum/bool 问卷渲染，Go/Node/Python 一套语义
交付包级运维	五件套交付 + compose 镜像行服务端改写；升级/回滚只认 registry tag，客户侧一条命令
验收回炉幂等闭环	UAT fail 携带交付上下文一键生成修复任务；内容标记去重，重复提交不重复建单
治理内建	审计导出（自身留痕）、成本分包（项目/周期）、密钥双钥轮换，均为平台原生能力



指标与成本分包：成功率趋势 / 项目周期聚合导出



平台治理：用户 / 角色 / 分页检索

五、质量与工程实践

- 后端测试覆盖率门禁（api 包 ≥78，实测 80%+），前端行覆盖 ≥90（实测 92%+）；
- CI 五 job：backend ∥ frontend ∥ e2e → security → build-push → deploy（串行控内存红线）；
- 数据库迁移版本表 + 每日备份 + 异地归档（S3/MinIO）；
- 全链路可观测：`forja_*` 指标、trace_id 贯穿、部署横幅与台账。

六、路线图（摘要）

版本	主题
v2.0	交付通道生产化（构建 / 制品 / 交付包 / 客户部署 / UAT）
v2.1	组织级模板库 + Go/Node 双栈
v2.2	Python 第三栈 + 模板参数化 + UAT 一键回炉 + 治理（审计导出/成本分包/密钥轮换）
后续	交付升级/回滚脚本化、监控与异地备份启用、私有化模型评估

📎 在线文档与最新截图：**https://iamartinlong.github.io/Forja/**
📎 本 PDF 随仓库发布：**https://iamartinlong.github.io/Forja/downloads/FORJA-项目介绍.pdf**